

# Certificati TLS, Secrets TLS e Network Policy

---

## Obiettivi di Apprendimento

Al termine di questa sezione il corsista sarà in grado di:

- Descrivere il sistema di Certificate Authority interno di OpenShift e l'annotazione per auto-provisioning dei certificati per i Service
  - Distinguere i tre tipi di terminazione TLS per le Route: `edge`, `passthrough`, `reencrypt`
  - Creare e gestire Secret di tipo TLS con `oc create secret tls`
  - Implementare Network Policy per il controllo del traffico Layer 4 tra Pod e namespace
  - Configurare una policy default deny-all e allow selettivo con `podSelector` e `namespaceSelector`
  - Spiegare le integrazioni TLS specifiche di ARO (Let's Encrypt, Azure Key Vault, NSG)
- 

## Teoria e Concetti Chiave

### Certificate Authority Interna di OpenShift

OpenShift crea e gestisce automaticamente una **CA interna del cluster** usata per:

- Certificare la comunicazione tra componenti interni (API server, etcd, Ingress, Service Mesh)
- Emettere certificati per i Service applicativi tramite annotazione

### Auto-Provisioning Certificati per Service

Tramite l'annotazione `service.beta.openshift.io/serving-cert-secret-name`, OpenShift emette automaticamente un certificato firmato dalla CA interna per un Service. Il certificato è rinnovato automaticamente prima della scadenza:

```
apiVersion: v1
kind: Service
metadata:
  name: myapp-service
  namespace: myproject
  annotations:
    service.beta.openshift.io/serving-cert-secret-name: myapp-tls # ← nome del
    Secret da creare
spec:
  selector:
    app: myapp
  ports:
    - port: 8443
      targetPort: 8443
```

Il Secret `myapp-tls` viene creato automaticamente nel namespace con `tls.crt` e `tls.key`. Il CA bundle è disponibile nel ConfigMap `openshift-service-ca/service-ca`.

## Tipi di Terminazione TLS per le Route

| Tipo        | Dove termina TLS                | Traffico interno              | Certificato necessario                | Uso tipico                                |
|-------------|---------------------------------|-------------------------------|---------------------------------------|-------------------------------------------|
| edge        | Ingress Controller (Router)     | HTTP non cifrato verso il Pod | Sul Router (può essere auto-generato) | App senza TLS interno, massima semplicità |
| passthrough | Il Pod/container stesso         | TLS end-to-end                | Nel Pod                               | Database, broker AMQP, app mTLS           |
| reencrypt   | Router (TLS esterno) + re-cifra | Nuovo TLS verso il Pod        | Sul Router + sul Pod                  | App con TLS interno, massima sicurezza    |

💡 **Suggerimento:** Per applicazioni che gestiscono dati sensibili in produzione su ARO, preferire **reencrypt** o **passthrough**. La terminazione **edge** espone il traffico in chiaro all'interno del cluster sul percorso Router → Pod.

## Secret di Tipo TLS

I Secret di tipo **kubernetes.io/tls** contengono un certificato X.509 e la chiave privata corrispondente:

```
# Creare un Secret TLS da file PEM esistenti
oc create secret tls myapp-tls \
  --cert=tls.crt \
  --key=tls.key \
  -n myproject

# Struttura del Secret:
# data:
#   tls.crt: <base64 del certificato PEM>
#   tls.key: <base64 della chiave privata PEM>
```

## Network Policy

**Network Policy** è un oggetto Kubernetes che controlla il traffico di rete a livello **Layer 4** (IP + porta) tra Pod. Su ARO (che usa OVN-Kubernetes come CNI), le Network Policy sono pienamente supportate.

## Comportamento Default

Senza Network Policy, tutto il traffico è **permesso** tra Pod in qualsiasi namespace. Con almeno una Network Policy che seleziona un Pod, solo il traffico esplicitamente consentito è ammesso:

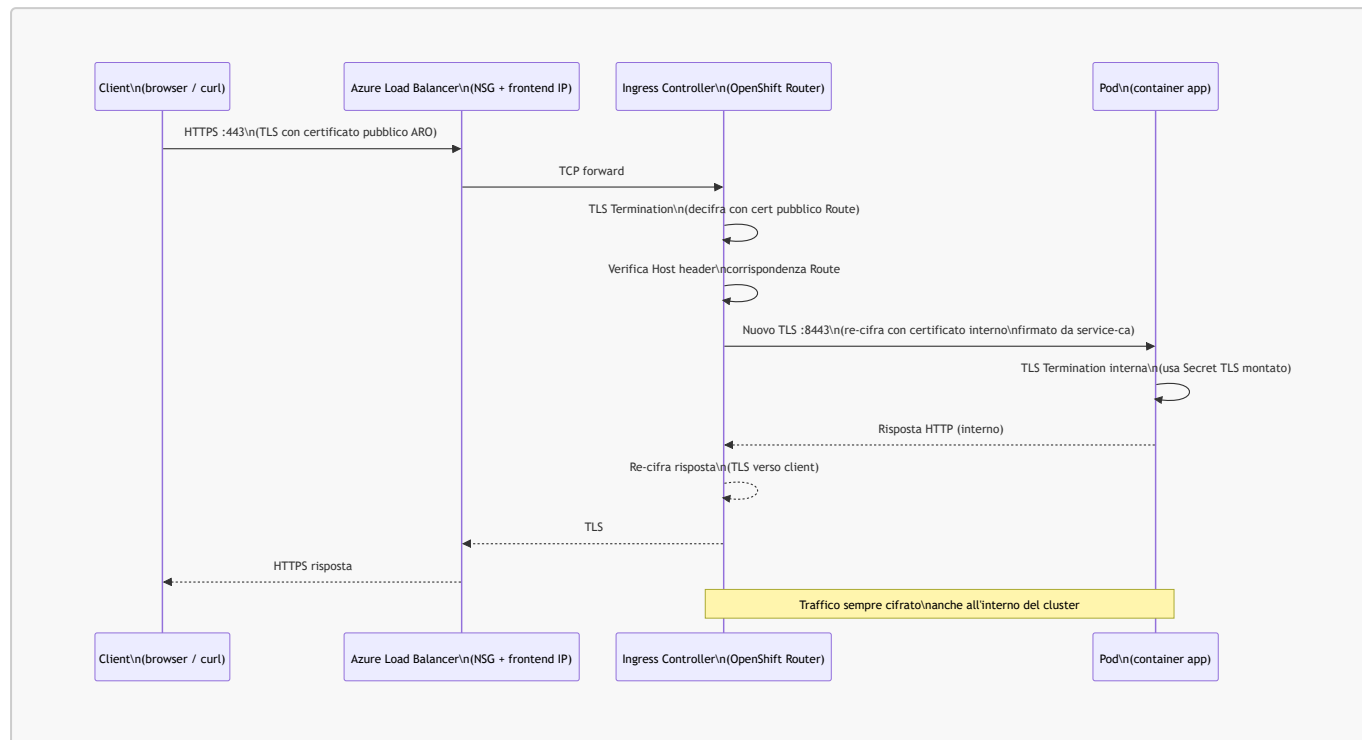
| Scenario                                                                     | Comportamento                               |
|------------------------------------------------------------------------------|---------------------------------------------|
| Nessuna NetworkPolicy nel namespace                                          | Tutto il traffico consentito (no isolation) |
| NetworkPolicy con <code>podSelector: {}</code> e nessun <code>ingress</code> | Default deny-all per i Pod selezionati      |
| NetworkPolicy con regole <code>ingress</code>                                | Solo il traffico dalle regole è ammesso     |

## Selettori nei NetworkPolicy Ingress/Egress

| Selettore                      | Descrizione                                    | Esempio                                                     |
|--------------------------------|------------------------------------------------|-------------------------------------------------------------|
| <code>podSelector</code>       | Seleziona Pod per label nello stesso namespace | Pod con <code>app: frontend</code>                          |
| <code>namespaceSelector</code> | Seleziona namespace interi per label           | Namespace <code>env: production</code>                      |
| <code>ipBlock</code>           | Range CIDR di IP (es. per database esterni)    | <code>10.0.0.0/8</code> eccetto<br><code>10.1.0.0/16</code> |

## Diagrammi

### Flusso TLS per una Route **reencrypt** su ARO



## Comandi Pratici con Output Atteso

```
# Creare una Route TLS edge (certificato auto-generato da OpenShift)
oc create route edge myapp-route \
  --service=myapp-service \
  --port=8080 \
  -n myproject

# Output atteso:
# route.route.openshift.io/myapp-route created

# Verificare l'hostname assegnato
oc get route myapp-route -n myproject -o jsonpath='{.spec.host}'
# Output atteso:
# myapp-route-myproject.apps.myaro.eastus.aroapp.io
```

```
# Creare una Route TLS reencrypt con certificato custom e certificato destinazione
oc create route reencrypt myapp-secure-route \
  --service=myapp-service \
  --cert=external-tls.crt \
  --key=external-tls.key \
  --dest-ca-cert=service-ca.crt \
  --port=8443 \
  -n myproject

# Output atteso:
# route.route.openshift.io/myapp-secure-route created
```

```
# Creare un Secret TLS da file esistenti
oc create secret tls myapp-tls-secret \
  --cert=tls.crt \
  --key=tls.key \
  -n myproject

# Output atteso:
# secret/myapp-tls-secret created

# Verificare il contenuto del Secret (senza esporre la chiave)
oc get secret myapp-tls-secret -n myproject \
  -o jsonpath='{.data.tls\.crt}' | \
  base64 -d | openssl x509 -noout -subject -dates

# Output atteso:
# subject=CN=myapp.myproject.svc
# notBefore=May 24 00:00:00 2026 GMT
# notAfter=May 24 00:00:00 2027 GMT
```

```
# Creare una Network Policy deny-all per il namespace myproject
oc apply -f - <<EOF
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: deny-all-ingress
  namespace: myproject
spec:
  podSelector: {}
  policyTypes:
    - Ingress
EOF
```

```
# Output atteso:
# networkpolicy.networking.k8s.io/deny-all-ingress created
```

```
# Verificare le Network Policy attive in un namespace
oc get networkpolicies -n myproject
```

```
# Output atteso:
# NAME                POD-SELECTOR  AGE
# deny-all-ingress   <none>        30s
# allow-frontend      app=backend    10s
```

```
# Testare la connettività tra Pod con una NetworkPolicy attiva
oc exec frontend-pod -n myproject -- \
  curl -s --max-time 5 http://backend-service:8080/health
```

```
# Output atteso (se consentito dalla NetworkPolicy):
# {"status":"ok"}
```

```
# Output atteso (se bloccato dalla NetworkPolicy):
# curl: (28) Connection timed out after 5001 milliseconds
```

```
# Ottenere il certificato dell'Ingress Controller ARO
oc get secret router-certs-default -n openshift-ingress \
  -o jsonpath='{.data.tls\.crt}' | \
  base64 -d | openssl x509 -noout -subject -issuer -dates
```

```
# Output atteso:
# subject=CN=*.apps.myaro.eastus.aroapp.io
# issuer=CN=Let's Encrypt Authority X3
# notBefore=May 01 00:00:00 2026 GMT
# notAfter=Jul 30 00:00:00 2026 GMT
```

```
# Verificare i certificati API e Ingress del cluster ARO tramite az CLI
az aro show \
  --name myaro \
  --resource-group my-resource-group \
  --query "{apiServer:apiserverProfile.url, ingressDomain:ingressProfiles[0].ip}" \
  -o table
```

# Output atteso:

| # ApiServer                               | IngressDomain  |
|-------------------------------------------|----------------|
| # -----                                   | -----          |
| # https://api.myaro.eastus.aroapp.io:6443 | 20.123.456.789 |

---

## Esempi YAML Completi

### Route TLS reencrypt + Secret TLS + NetworkPolicy Completa

```
# Secret TLS con certificato pubblico per la Route (firmato da CA pubblica o Let's
Encrypt)
apiVersion: v1
kind: Secret
metadata:
  name: myapp-external-tls
  namespace: myproject
  labels:
    app: myapp
type: kubernetes.io/tls # ← tipo obbligatorio per Secret TLS
data:
  tls.crt: <base64-encoded-certificate> # ← certificato PEM (inclusa la chain
intermedia)
  tls.key: <base64-encoded-private-key> # ← chiave privata PEM
---
# Route TLS di tipo reencrypt
# Termina TLS esternamente con il certificato pubblico,
# re-cifra il traffico verso il Pod con il certificato interno (service-ca)
apiVersion: route.openshift.io/v1
kind: Route
metadata:
  name: myapp-secure
  namespace: myproject
  labels:
    app: myapp
spec:
  host: myapp.apps.myaro.eastus.aroapp.io # ← hostname pubblico (o omettere per
auto)
  port:
    targetPort: 8443 # ← porta del Service verso il Pod
  to:
    kind: Service
    name: myapp-service
    weight: 100
  tls:
    termination: reencrypt # ← edge | passthrough | reencrypt
    # Certificato esterno (verso i client)
    certificate: | # ← PEM inline (alternativa al Secret)
      -----BEGIN CERTIFICATE-----
      ...
      -----END CERTIFICATE-----
    key: |
      -----BEGIN PRIVATE KEY-----
      ...
      -----END PRIVATE KEY-----
    # CA del certificato interno del Pod (service-ca di OpenShift)
    destinationCACertificate: |
```



```

-----BEGIN CERTIFICATE-----
...
-----END CERTIFICATE-----
insecureEdgeTerminationPolicy: Redirect # ← redirect HTTP → HTTPS
---
# NetworkPolicy: deny-all ingress di default per tutti i Pod del namespace
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: default-deny-all-ingress
  namespace: myproject
  labels:
    policy-type: default
spec:
  podSelector: {} # ← {} = seleziona TUTTI i Pod del namespace
  policyTypes:
    - Ingress # ← controlla solo il traffico in ingresso
  # Nessuna regola ingress = deny all per i Pod selezionati
---
# NetworkPolicy: permetti traffico dal namespace dell'Ingress Controller al Pod myapp
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: allow-ingress-controller
  namespace: myproject
spec:
  podSelector:
    matchLabels:
      app: myapp # ← si applica solo ai Pod con label app=myapp
  policyTypes:
    - Ingress
  ingress:
    - from:
        # Permette traffico dal namespace openshift-ingress (Router)
        - namespaceSelector:
            matchLabels:
              kubernetes.io/metadata.name: openshift-ingress
      ports:
        - protocol: TCP
          port: 8443 # ← porta del container (non del Service)
---
# NetworkPolicy: permetti traffico dal Pod frontend al Pod backend nello stesso namespace
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: allow-frontend-to-backend
  namespace: myproject
spec:
  podSelector:

```

```
    matchLabels:
      app: backend                                     # ← policy applicata ai Pod backend
  policyTypes:
  - Ingress
  ingress:
  - from:
      # Permette solo dal Pod frontend nello stesso namespace
      - podSelector:
          matchLabels:
            app: frontend
        ports:
        - protocol: TCP
          port: 8080
  ---
# NetworkPolicy: deny-all egress (blocca tutto il traffico in uscita)
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: default-deny-all-egress
  namespace: myproject
spec:
  podSelector: {}
  policyTypes:
  - Egress
  egress:
    # Permette solo il traffico DNS (necessario per la risoluzione dei nomi)
    - ports:
        - protocol: UDP
          port: 53
        - protocol: TCP
          port: 53
```

## Note Specifiche per Azure ARO

### Certificato Let's Encrypt per il Dominio Wildcard ARO

Su ARO, il dominio wildcard `*.apps.<cluster>.<region>.aroapp.io` è fornito con un certificato **Let's Encrypt** valido 90 giorni, rinnovato automaticamente da ARO:

```
# Verificare il certificato Ingress wildcard di ARO
oc get secret router-certs-default -n openshift-ingress \
  -o jsonpath='{.data.tls\.crt}' | base64 -d | \
  openssl x509 -noout -subject -issuer -enddate

# Sostituire il certificato Ingress con un certificato custom (es. aziendale)
# 1. Creare il Secret con il certificato custom
oc create secret tls custom-ingress-cert \
  --cert=company-wildcard.crt \
  --key=company-wildcard.key \
  -n openshift-ingress

# 2. Aggiornare l'IngressController per usare il Secret custom
oc patch ingresscontroller default \
  -n openshift-ingress-operator \
  --type=merge \
  --patch='{ "spec": { "defaultCertificate": { "name": "custom-ingress-cert" } } }'
```

## Azure Key Vault per Certificati TLS su ARO

Su ARO, è possibile integrare **Azure Key Vault** come source of truth per i certificati TLS tramite il **Secrets Store CSI Driver** (già visto nel Modulo 7):

```
# Creare un SecretProviderClass per un certificato in AKV
cat <<EOF | oc apply -f -
apiVersion: secrets-store.csi.x-k8s.io/v1
kind: SecretProviderClass
metadata:
  name: tls-cert-from-akv
  namespace: myproject
spec:
  provider: azure
  parameters:
    usePodIdentity: "false"
    useVMManagedIdentity: "true"
    userAssignedIdentityID: "$(az identity show --name myaro-identity -g myaro-rg
--query clientId -o tsv)"
    keyvaultName: "myaro-keyvault"
    objects: |
      array:
        - |
          objectName: myapp-tls-cert
          objectType: secret          # ← tipo "secret" in AKV per certificati
          objectVersion: ""
          secretObjects:
            - secretName: myapp-tls-from-akv    # ← Secret Kubernetes generato
              automatically
              type: kubernetes.io/tls
              data:
                - objectName: myapp-tls-cert
                  key: tls.crt
EOF
```

## NSG Azure + Network Policy Kubernetes: Doppio Strato

Su ARO, la sicurezza di rete opera su **due livelli indipendenti**:

| Livello           | Tecnologia                       | Scope                                | Gestione              |
|-------------------|----------------------------------|--------------------------------------|-----------------------|
| L3/L4 perimetrale | Azure NSG                        | Traffico verso/dal cluster (esterno) | Azure Portal / CLI    |
| L3/L4 interno     | Kubernetes NetworkPolicy (OVN-K) | Traffico tra Pod e namespace         | <code>oc apply</code> |

**⚠ Attenzione:** Le Network Policy Kubernetes controllano **solo** il traffico tra Pod all'interno del cluster. Per bloccare il traffico proveniente dall'esterno del cluster (Internet, peering VNet), usare le **Azure NSG** sul subnet del nodo ARO.

```
# Verificare le NSG associate al subnet dei worker ARO
az network nsg list \
  --resource-group "$(az aro show --name myaro -g myaro-rg --query
clusterProfile.resourceGroupId -o tsv)" \
  --query "[].{Name:name, Location:location}" \
  -o table

# Output atteso:
# Name                               Location
# -----
# aro-worker-nsg                     eastus
# aro-master-nsg                     eastus
```

## Riepilogo dei Punti Chiave

- OpenShift include una **CA interna** che firma automaticamente certificati per i Service tramite l'annotazione `service.beta.openshift.io/serving-cert-secret-name`
  - Le Route TLS supportano tre tipi di terminazione: `edge` (TLS termina al Router), `passthrough` (TLS end-to-end nel Pod), `reencrypt` (TLS esterno + TLS interno, massima sicurezza)
  - I Secret di tipo `kubernetes.io/tls` contengono `tls.crt` e `tls.key` in formato PEM codificato Base64
  - Le **Network Policy** operano a Layer 4; senza policy, tutto il traffico tra Pod è permesso; la prima policy che seleziona un Pod attiva l'isolation mode (deny-all implicito per i tipi di traffico selezionati)
  - Il pattern consigliato: creare prima una policy `deny-all` per il namespace, poi aggiungere policy `allow` selettive
  - Su ARO, il certificato wildcard `*.apps.*` è Let's Encrypt con rinnovo automatico; può essere sostituito con un certificato aziendale aggiornando `IngressController.spec.defaultCertificate`
  - Su ARO, la sicurezza di rete opera su due livelli: **NSG Azure** (esterno al cluster) e **Network Policy Kubernetes** (interno al cluster)
-

## Domande di Verifica

**1. Quale tipo di terminazione TLS garantisce che il traffico sia cifrato sia all'esterno che all'interno del cluster (Router → Pod)?**

- A) `edge`
- B) `passthrough`
- C) `reencrypt` ✓
- D) `mutual`

**2. Qual è l'effetto di creare una NetworkPolicy con `podSelector: {}` e `policyTypes: [Ingress]` senza regole `ingress` in un namespace?**

- A) Nessun effetto — la policy è vuota e non fa nulla
- B) Permette tutto il traffico ingress verso tutti i Pod del namespace
- C) Blocca tutto il traffico ingress verso tutti i Pod del namespace ✓
- D) Elimina tutte le Network Policy esistenti nel namespace

**3. Quale annotazione su un Service OpenShift fa sì che venga automaticamente creato un Secret con il certificato TLS firmato dalla CA interna del cluster?**

- A) `service.openshift.io/auto-tls: "true"`
- B) `service.beta.openshift.io/serving-cert-secret-name: <nome-secret>` ✓
- C) `openshift.io/tls-secret: <nome-secret>`
- D) `kubernetes.io/tls-provision: "auto"`

**4. Su ARO, quale strumento Azure gestisce il traffico di rete in ingresso proveniente dall'esterno del cluster (es. da Internet)?**

- A) Kubernetes NetworkPolicy
- B) OpenShift EgressFirewall
- C) Azure Network Security Groups (NSG) ✓
- D) OpenShift Ingress Controller

**5. Un'applicazione su ARO usa una Route di tipo `passthrough`. Dove deve essere configurato e gestito il certificato TLS?**

- A) Sul Router di OpenShift (Ingress Controller), nel Secret `router-certs-default`
- B) In un Secret montato nel Pod, letto direttamente dall'applicazione ✓
- C) Nella Route stessa, nel campo `spec.tls.certificate`
- D) In Azure Key Vault, accessibile solo tramite AKV Reference